

LESSON GUIDE

Technology Plus

Unit 6: Database

5.3.1 – Database Structures

Lesson Overview:

In this lesson, students will compare and contrast various database structures and explore the difference between relational and non-relational databases.

Students will:

- Compare and contrast structured, semi-structured, and non-structured data.
- Explore the key differences between a relational database and non-relational database.

Guiding Question: What are the three database structures, and how do they differ from one another?

Suggested Grade Levels: 8-12

Technology Needed: Yes

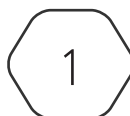
Approximate Time Required: 85-125 minutes

Standards:

CompTIA Tech+ FC0-U71 Objective

- 5.3 – Compare and contrast various database structures.
 - Structured vs. semistructured vs. non-structured
 - Relational databases
 - Schema
 - Tables
 - Rows/records
 - Fields/columns
 - Primary key
 - Foreign key
 - Constraints
 - Non-relational databases
 - Key/value databases
 - Document databases

This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).



CYBER.ORG
THE ACADEMIC INITIATIVE OF THE CYBER INNOVATION CENTER

Copyright © 2025 Cyber Innovation Center
All Rights Reserved. Not for Distribution.

Lesson Background

Background Information:

This lesson is built upon 5.2.1 – Database Fundamentals. Students will learn about the three different structures of a database and compare them to one another. Database structures will fit into a relational database or non-relational database. In this lesson, students learn more about schema, primary keys, foreign keys, and other constraints that structure a relational database. They will also learn about the two common types of non-relational databases. The lab in this lesson will take students through the process of creating tables within a database and designing its schema by defining data types and constraints which allow for complex querying and quick data retrieval. The activity provides students with a non-technical insight into utilizing a document database to handle data that has varying formats.

Materials Needed:

Materials per Class	• Lesson Guide • Lesson Slides 5.3.1 – Database Structures • Lab Slides 5.3.1 – SQL Structures • Activity Slides 5.3.1 – Document Database Simulation
Materials per Student	• Student Notes 5.3.1 – Database Structures • Student Handout 5.3.1 – Database Structures • 2 Index Cards • Assessment 5.3.1 – Database Structures (optional)

Lesson Background

Cyber Terms:

Term	Definition
structured data	data that is organized in a specific, predefined manner making it easy to read and understand
semi-structured data	data that does not conform to a rigid schema but still has some level of organization or structure
non-structured data	data that does not have a predefined structure making it challenging to collect, process, and analyze
relational database	a type of database that stores data in a structured format using tables made of records and fields
schema	the rules or structure of a database
data model	a blueprint used to provide a structured framework, or schema, for a database
logical data model	outlines the structure of a relational database, also known as logical schema
physical data model	defines data is represented within a relational database management system, also known as physical schema
tables	structure data into records and fields within a relational database
records	a collection of fields that contain data about a given entity, also known as a row
fields	store a specific type of data for all records, also known as a column
primary key (PK)	uniquely identifies each record in a table, also connects multiple tables within a relational database
foreign key (FK)	a field in one table that is the primary key of another table, creating a link between different tables and their data
constraints	rules applied to tables within a database to enforce data integrity
non-relational database	a type of database that provides various ways to store and manage data that is not structured
key-value database	a non-relational database that stores data as a collection of key-value pairs
document database	a non-relational database that stores data in the form of documents

5.3.1 – Database Structures

Guiding Question: What are the three database structures, and how do they differ from one another?

Lesson Launch (5 minutes)

Structure: To Be or Not to Be

- Tell students that structure provides organization or a specific pattern to follow.
- Prompt students to think about how structure at school is different from structure during the weekend.
- Ask: Is structure necessary?
- Students should form a conclusion that structure can be beneficial but is not needed in every situation.
- Tell students that databases also benefit from having various structures. Database structures are chosen to meet the needs of an organization and the types of data they need to store or manage.
- Remind students that we previously learned about the fundamentals of a database.
- In this lesson they will learn more about the structure of a database and how it varies depending on the data set it is created for.

Database DNA (25-35 minutes)

- Distribute Student Handout 5.3.1 – Database Structures and encourage students to fill in the answers during the lesson.

Structured Data vs. Semi-structured Data vs. Non-structured Data

- Introduce **structured data**, its definition, advantages and disadvantages.
 - Share the examples below and point out the graphic on slide 4, each shape is the same.
 - When going to a grocery store each item is different but can share categories (fields), such as name, food group, and price. When storing structured data each item will share the same categories (fields) and have a data value for each.
 - Customer databases, inventory management systems, or financial records are structured data.
- Introduce **semi-structured data**, its definition, advantages, and disadvantages.
 - Semi-structured data is data that has been tagged with metadata. Metadata is data about data. It provides structure to unstructured data by providing context. i.e. metadata on an image may be the date the image was taken, the location, or specific keywords about what is in the picture.
 - Share the examples below and point out the graphic on slide 5, there are two different shapes and two different colors. Overall, the square is still the primary shape.

- When receiving an email, there is a sender, a subject line, and a message body. Emails can be tagged for a specific category – social updates, important, to-do, create your own group. If an email is tagged, it can be easier to find (or query) later. If someone does not tag their email and cannot remember the sender or subject line, it can become difficult for them to find what they may be looking for. Tagged emails are considered semi-structured data because they have some structure but cannot fit neatly into rows and columns.
- JSON or XML documents, email data, and web data are semi-structured data.
- Introduce **non-structured data**, its definition, advantages, and disadvantages.
 - Share the examples below and point out the graphic on slide 6, there are different shapes, colors, and it is not neat.
 - A folder on a computer named 'Vacation' has photos, a text document of places to visit, a PDF with directions to a hotel, and emails from a travel agent. Although each file is about the vacation, there is no clear structure amongst the files – the data is unstructured.
 - Mention that the data has structure within itself (i.e., a picture has a time, date, and location of when it was captured) but is considered unstructured because it does not fit neatly into a defined schema.
 - A collection of social media posts, video and audio files, text documents and reports are non-structured data.
- Mention the differences between semi- and non-structured data: Semi-structured data can be queried (somewhat). Both do not fit neatly into rows and columns. Non-structured data can be photos, emails, documents, phone numbers, etc. Semi-structured data is non-structured data that has been tagged to define some of its characteristics.

Relational Databases

- Introduce a **relational database** and its main elements.
 - Discuss **schema** and how it can define a database.
 - Discuss how **data models** affect schema design and introduce the three types of data models.
 - Mention that conceptual data models are a great starting point for brainstorming the framework of a database. Logical data models lead to outlining tables, fields, primary keys, foreign keys, constraints, and more for the database. The physical data model defines the code that will create a database in the designated management system. Each model is built on top of each other.
 - The example data models for this section will be on a school database.
- Introduce the **conceptual data model** showing the relationship between tables in a school database.
 - Tables: Students, enrollment, courses, grades, instructors.
 - Relationships: Students to enrollment, enrollment to courses, instructors to courses, courses to grades, grades to students.
- Introduce the **logical data model** showing tables, fields, and relationships.

- The tables and relationships match from the conceptual data model. Point out the field names, and the fields that are identified as primary key (PK) or foreign key (FK) .
- Each table has a primary key as a unique identifier.
- Introduce the **physical data model** showing the code that can be put into a database management system (DBMS) to set up the school database.
 - The code includes the table name, fields, data types, and constraints.
 - This specific physical data model can be used to create tables in the database, the creator would need to put a comma after each data value, and a semi-colon at the end to complete the transaction.
- Logical data models outline the database, and physical data models create the database. Logical data models should be created first and any changes made to the physical data model should be reflected in the logical data model.
- Discuss **tables** and how they structure data into rows (records) and columns (fields) in a relational database.
 - The image on slide 13 shows the code for creating a table using SQLite in the Linux terminal. The code defines the table name, the name of each field, their data types, and constraints. Each field is defined by a comma and the table entry is completed with a semicolon.
- Discuss **records** and **fields**.
 - Remind students that rows and columns in a database are known as records and fields.
 - Mention that every record in a database is unique and identifiable by its primary key. In the image on slide 14, “student_id” is the primary key for each record. Each record represents a student and the data specific to them.
 - Mention that fields define the data type and are named to describe the data it is holding. In the image on slide 14, the “last_name” field holds the last name for each record in the database.
- Introduce **primary key** and its definition.
 - Mention that a table cannot be relational in a database if it does not have a primary key. Sometimes data in a table can repeat such as names, grade level, birth dates, and more. Defining a primary key allows each record to stay unique no matter what.
- Introduce **foreign key** and its definition.
 - Mention that unlike primary keys, foreign keys are not required but ensure data consistency across tables.
- Primary and foreign keys can have different relationships.
 - In a one-to-one relationship, the unique field is often the primary key in both tables. In a one-to-many relationship a primary key in one table will often appear as a foreign key in another table.
 - The image on slide 17 shows the logical data model of the school database with the relationships between tables defined as one-to-many relationships. Point out that each table is only connected to a table that has its primary key defined as a foreign key.
- Introduce **constraints**, their definition, and the types of constraints.

- Reference the image on slide 13, students saw the primary key, not null, and unique constraints defined in the table creation code.

Non-Relational Databases

- Introduce a **non-relational database**, its definition, and the two common types: key-value databases and document databases.
 - Introduce a **key-value database**, its definition, key characteristics, and advantages.
 - Introduce a **document database**, its definition, key characteristics, and advantages.
 - Nested files and arrays are ways to store related data within a single document making it easier to retrieve, update, or delete data without requiring multiple queries or join queries like in a relational database.
 - Remind students that join queries allow users to retrieve data across different tables.
 - Non-relational databases provide various ways to store and manage data that is not structured. They do not conform to schema, allow flexibility for varying data structures, and offer high scalability for growing data sets.

Lab: 5.3.1 SQL Structures (30-45 minutes)

- This lab requires access to the CYBER.ORG Range, the Ubuntu Linux Machine, and Lab Slides 5.3.1 – SQL Structures.
- The lab can be performed as a class activity or an independent student assignment.
- Introduce the guiding question and objectives for the lab.
- Students will use the Ubuntu Linux Machine to create tables, design schema, and run queries on a database using SQLite3.
- Discuss SQL Key Terms students will use in the lab.
- In the Ubuntu VM, start by opening the terminal window. Students should navigate to the directory containing the csv files they will import into the tables they create in the database. After the prompt changes to `sqlite>`, the terminal will still create and import files from the Database_Structures-lab folder.
- Students should complete the lab portion of their Student Handout.
- This lab builds upon knowledge from the lab in lesson 5.2.1 – Database Fundamentals.

Helpful Tips

- Remind students to take their time when typing in the terminal during this lab. Missing even one letter can cause them to retype an entire command.
- When defining a field as **PRIMARY KEY**, it also applies UNIQUE and NOT NULL constraints to the field.
- As students create tables in the terminal, it is not necessary for them to press [ENTER] after each field. Images show this for format purposes. Encourage students to do what works for them.
- Students should put a semicolon (;) at the end of each query before hitting [ENTER]; otherwise the

query will not run.

- If students make a mistake when creating a table, they can use **DROP TABLE table_name;** to delete the table.
- If .mode csv is not used before importing the file, it will not import correctly, and students will not be able to format the data to columns.
- Remind students they can cycle through previous commands by pressing the up arrow on the keyboard.
- If students upload the wrong file to a table, they can use the **DELETE FROM table_name;** to remove the data without deleting the table.
- If **PRAGMA foreign_keys = ON;** is not used, the database will not follow the constraints set in the schema design and will compromise data integrity.
- If 'foreign' is misspelled when enabling foreign_keys, they will not be enabled, and data integrity will be compromised.
- When the UPDATE query is used to change the **DATE** data type in the terminal, single quotes (') are needed around date or the data will not update correctly. i.e. '10/22/2025'.
- When running an INNER JOIN query, values "a", "b", and "c" are assigned to tables to specify where data is pulled from for certain fields.
- When using text, or a string, in a query – single quotes are needed. i.e. 'Hello World'.

Put It All Together

- When defining a field as primary key, what other constraints are applied to the field?
Defining a field as primary key applies the unique and not null constraints.
- What command is used to save changes to a database?
The COMMIT command is used to save changes to a database.
- What query did we run to select records that have matching values across different tables?
We ran a JOIN query to select records that have matching values across different tables. Students could also specify with the INNER JOIN query.
- What is the purpose of foreign keys in a database?
Foreign keys in a database create links or relationships between two different tables. A foreign key in one table is the primary key in another table.

Activity: Document Database Simulation (20-30 minutes)

- In this activity, students will use index cards to create a non-relational document database. Each index card will represent a document, and all the index cards together will represent the entire database.
- Each student should receive two index cards; on each index card the student will create an animal profile. Encourage students to be creative as they make their pet profile. There is no rigid schema – the possibilities are endless!
- Tell students from now on they will refer to their index cards as documents.

- During the query simulation, students will have to manually search their documents. If their pet profile matches the query, have those students come to the front of the room. This will allow students to visibly see the difference in their documents.
- Students should only bring the document that matches the query to the front of the room. Documents will be counted, and students will record the answer on their handout.
- Students will update their documents along with slides 5-7.
- Students will have another query simulation based on their updated documents. The same process will be followed as before to ensure accurate document count.
- Student Handout questions below:

Question 24-30 are query-based and answers depend on what students wrote in their pet profiles.

1. How many pets in the database are under three years old?
2. How many pets in the database are not vaccinated?
3. How many pets in the database belong to the “Cat” species?
4. How many pets in the database have an adoption fee?
5. How many pets in the database have an adoption fee under \$100?
6. How many pets in the database are spayed/neutered?
7. How many pets in the database have behavior notes on their profile?
8. What are the advantages of using a non-relational database over a relational database for this data set?

A non-relational database allows this data set to have schema flexibility in each document and there are no restrictions on what can be included in each document.

Putting It All Together (5-10 minutes)

- Review what students learned today by discussing the following questions as a class.
- What are the three database structures?

The three database structures are structured, semi-structured, and non-structured.

- How are the database structures similar?

All data is stored in a database that fit their structure. Semi-structured data and non-structured data both do not follow a predefined schema and are easily scalable as data grows.

- How are the database structures different?

Structured data follows a rigid predefined schema, can run complex queries, and enforces data integrity. Semi-structured data has a flexible schema and can tag data for simple queries. Non-structured data has no schema and does not enforce data integrity.

- What is the difference between a relational database and a non-relational database?

A relational database stores data in a structured format using tables made of records and fields and enforces data integrity through schema and constraints. A non-relational database provides various

ways to store and manage data that is not structured and can easily change to handle varying data types.

- What are the main elements of a relational database?

The main elements of a relational database are schema, tables, records, fields, primary keys, foreign keys, and constraints.

- What are the two common types of non-relational databases?

The two common types of non-relational databases are key-value databases and document databases.